上海交通大学
SHANGHAI JIAO TONG UNIVERSITY

SJTU

数据预处理
——pandas简介

上海交通大学RFID与物联网研究所

饮 水 思 源 • 爱 国 荣 校

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY

目 录



1 预处理工具

2 数据预处理

3 示例

常见工具

① **Excel**：最常见的工具之一，用于小规模数据处理。

② **Kettle**：支持从各种数据源抽取数据，进行转换和加载，具有图形化界面和命令行接口，适用于大规模数据处理和批量任务自动化。

③ **Apache Nifi**：用于自动化数据流和数据集成的工具，具有图形化界面，可以帮助用户自动化数据流的构建、管理、调度等任务。

④ **Pandas**：Python中最常用的数据处理库，支持对数据进行读取、写入、清洗、筛选、分组、去重、合并、填充缺失值、格式转换等操作。

Pandas介绍



Pandas 是一个开源的 Python 数据分析库，广泛应用于数据处理、清洗、分析以及数据可视化等领域。它为数据科学家和分析师提供了高效的数据结构和数据分析工具，尤其擅长处理表格数据。Pandas 的核心数据结构是 **DataFrame**，可以将其看作是一个二维的表格数据结构，类似于 Excel 中的表格。

- DataFrame 是 Pandas 中的另一个核心数据结构，类似于一个二维的表格或数据库中的数据表。
- DataFrame 是一个表格型的数据结构，它含有一组有序的列，每列可以是不同的值类型（数值、字符串、布尔型值）。
- DataFrame 既有行索引也有列索引，它可以被看做由 Series 组成的字典（共同用一个索引）。
- DataFrame 提供了各种功能来进行数据访问、筛选、分割、合并、重塑、聚合以及转换等操作。
- DataFrame 是一个非常灵活且强大的数据结构，广泛用于数据分析、清洗、转换、可视化等任务。

Pandas安装



简单容易

```
pip install pandas
```

- Tip：需要具有python和pip环境，python安装可参考：https://zhuanlan.zhihu.com/p/104502997

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY

目 录



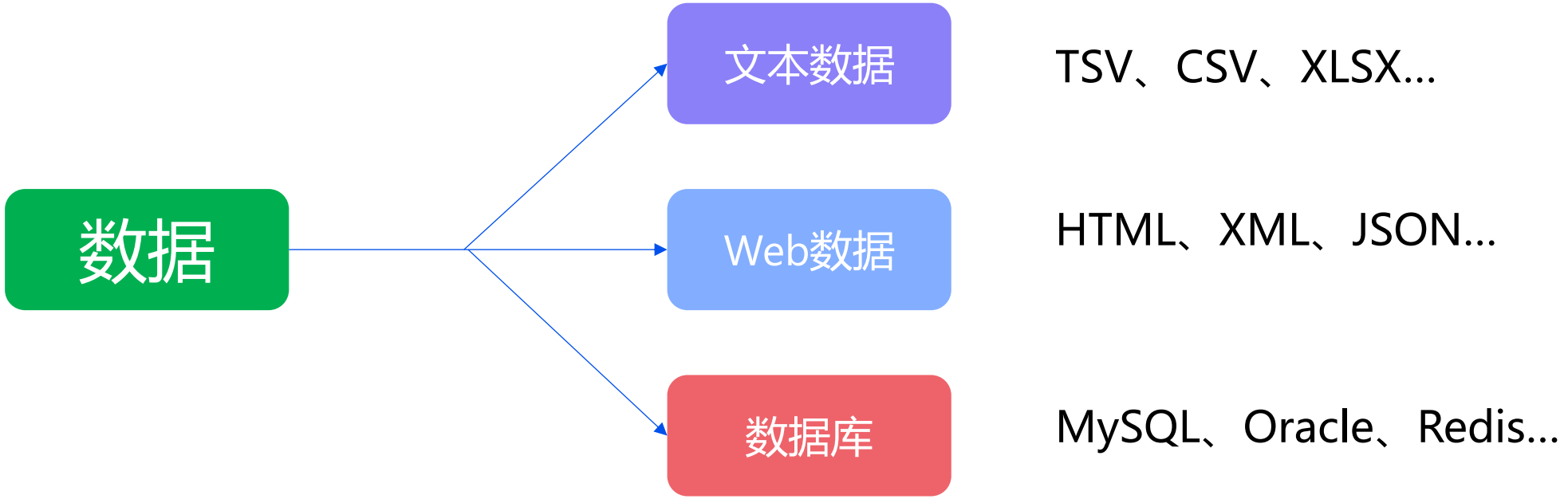
1 预处理工具

2 数据预处理

3 示例

数据抽取

数据抽取主要是从一个或多个数据源中获取所需的数据。数据抽取的数据源可以是文本数据、Web数据以及数据库数据。



数据清洗与校验



数据清洗是一项复杂且繁琐的工作，同时也是整个数据分析过程中最为重要的环节。数据清洗的目的在于提高数据质量，将脏数据（脏数据在这里指的是对数据分析没有实际意义、格式非法、不在指定范围内的数据）清洗干净，使原数据具有完整性、唯一性、权威性、合法性、一致性等特点。常见的数据清洗操作包括重复值的处理、缺失值的处理、异常值的处理等操作，同时，为了保证数据的有效性，少不了数据校验操作。

数据清洗与校验

- **数据去重**

1. 完全去重指的是消除完全重复的数据，这里提到的完全重复数据指的是数据表记录字段值完全一样的数据。例如，现在有两个表格分别记录的不同年份的用户信息，现要求合并统计所有用户信息，发现合并后的表格存在完全重复的数据,为了便于后期更加方便地使用这些用户数据，通常情况下会对数据进行去重操作。

2017年部分用户名单

Name	Gender	City
zhangsan	male	beijing
liwu	male	nanjing
hanmeimei	female	hebei
wangliu	male	henan

2018年部分用户名单

Name	Gender	City
zhangsan	male	beijing
lisi	male	nanjing
hanmeimei	female	hebei
wangwu	male	henan
xiaoming	female	sichuan

Name	Gender	City
zhangsan	male	beijing
liwu	male	nanjing
hanmeimei	female	hebei
wangliu	male	henan
zhangsan	male	beijing
lisi	male	nanjing
hanmeimei	female	hebei
wangwu	male	henan
xiaoming	female	sichuan

数据清洗与校验

- **数据去重**

2.不完全去重：数据清洗过程中，所有字段值都相等的重复值是一定要剔除的。根据不同的业务场景，有时还需要选取其中若干字段进行 去重操作。

数据清洗与校验

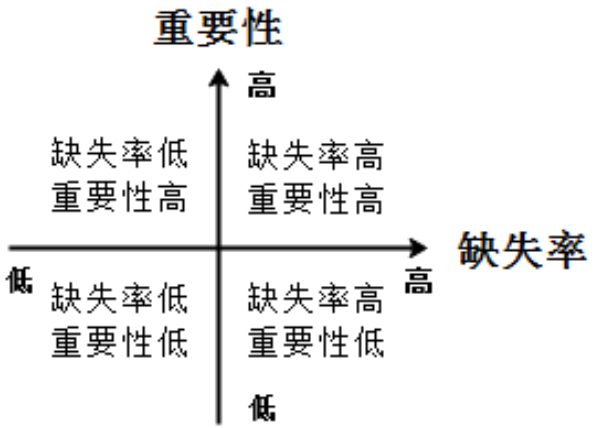
- **缺失值处理**

缺失值是指数据集中某个或某些属性的值是不完整的，产生的原因主要有人为原因和机械原因两种，其中机械原因是由于机器故障造成数据未能收集或存储失败，人为原因是由主观失误或有意隐瞒造成的数据 缺失。

数据清洗与校验

• 缺失值处理

制定合理的缺失值数据处理策略，不仅可以提升缺失值数据处理的效率，还可以使处理后数据的可靠性得到保证，提高最终分析结果的准确性。缺失值的处理方法很多，这里建议大家在清洗缺失值时，首先计算数据源字段缺失值比例，之后根据数据缺失率和重要性，指定不同的策略。下面，通过一张图来描述缺失数据的缺失率以及重要性划分的四种情况，具体如图所示。



数据清洗与校验

• 缺失值处理

根据缺失值的范围制定对应的策略，通常情况下采用的策略如表所示。

缺失值范围	对应策略
重要性高、缺失率高的数据	(1) 尝试从其它渠道获取数据进行补全 (2) 通过对其它字段的数据进行分析、计 算等方式获取合理值进行补全 (3) 去除字段但要在结果中进行标注
重要性高、缺失率低的数据	(1) 通过对字段自身的数据进行分析、计 算等方式获取合理值进行补全 (2) 通过自身的经验与业务知识对缺失值 数据进行人为补全
重要性低、缺失率高的数据	直接去除该字段
重要性低、缺失率低的数据	可以不去处理或者进行简单的填充

数据清洗与校验

• 异常值处理



异常值是指样本中的个别值，其数值明显偏离它所属样本的其余观测值，这些数值是不 合理的或错误的。

不少人在处理异常值时，习惯于使用简单粗暴的删除方法，但是这并不适用于处理所有 异常值，若是通过删除的方法处理异常值，有时会使你错过真正的规律。如果想要删除数据，一定需要有合理的理由。当出现异常值时，需要先搞清楚是什么原因导致的异常值， 然后考虑如何去处理。

数据清洗与校验

• 异常值处理

1. 删除包含异常值的记录：如果异常值是由于输入或测量数据不正确而造成的，则很明显就能看出这一类数据是错的。例如，假设有一个关于体重的数据集，其中女性的体重记录为19斤，通常这样的体重对于 一个女性来说是不正常的，她的真实体重可能是119斤或190斤等，由于我们无法确定是哪个值，因此这一类数据可以直接删除。

极端异常值会对平均值产生很大影响，但不会影响中位数。因此， 如果只计算中位数，则可以包含异常值。如果异常值过于极端而不可信 （例如，可能由于测量误差），则应该将其排除；如果异常值是合理的 ，则需要进行分析是否有异常值数据。如果这两种类型的数据分析得出的结果一致，则可以删除该异常值。

数据清洗与校验

• 异常值处理

不宜删除异常值的情况，具体如下：

（1）通常情况下，数据中出现的异常值较少。如果采集的数据中有超过30%的异常值数据，那么意味着需要进一步研究数据。

（2）如果异常值存在且代表了一种真实存在的现象，那就不可随意删除。例如，调查100个村的胃癌发病率，可能确实有个别村庄的发病率远远高于其它村，这时就不能随意删除，而是要把这些异常点纳入，重新拟合模型，研究其规律。

（3）分析数据的结果至关重要，因此即使很小的变化也会很重要。例如，可以更好地放弃关于人们最喜欢电视节目的异常值，而不是放弃关于飞机封条失效的温度。

数据清洗与校验

• 异常值处理

2. 修补异常值：通过直接删除的方式处理异常值，虽然是最直接方法的方法，但是会减少数据样本，因此在数据集小的情况下，减少数据样本会对结果产生影响；在含有较多异常值的数据集中，大量的删除异常值也会对结果产生影响。因此，当异常值没有可研究性的情况下，应该对这些异常值进行修补处理。

修补异常值的方式主要有两种，即改异常值和替换异常值。

数据清洗与校验

• 数据检验

1. 数据一致性处理：

数据的一致性有三种类型，即强一致性、弱一致性以及最终一致性， 具体介绍如下：

- 强一致性是指对数据完成更新操作之后，所有客户端访问到的数据均为更新之后的数据，这样可以保证客户端获取的是最新数据，但是若要达到强一致性，将会降低性能。
- 弱一致性是指当数据完成更新操作之后，系统并不保证所有的客户端访问到的数据都是最新数据，但是会尽量保证在某个时间（如秒级或分钟 级）内，让数据达到一致性状态。
- 最终一致性是弱一致性的一种特例，当对数据更新完之后，保证没有后续更新的前提下，系统最终返回的是上一次更新操作的值。

数据清洗与校验

• 数据检验

2. 数据规范化处理：

为了提高数据的可读性及合理性，企业会要求数据遵守一定的规范， 具体规范如下：

- 电子邮箱的地址必须是有效的格式；
- 输入的数据都必须是大写/小写；
- 日期必须是dd-mm-yyyy的格式；
- 电话号码必须是xxx-xxxx-xxxx的格式；
- 用户的年龄必须大于18岁；
- 数值不能超过预定义的值。

综上所述，这些规范都有一个共同点，即检查数据都必须遵守预定义 的业务规则，找出不符合业务规则的数据。

数据转换



数据转换是数据清洗过程的重要步骤之一，它的主要任务是进 行不一致数据的转换、数据粒度的转换、以及一些商务规则的计算。

数据转换

- **多源数据合并**

随着信息技术的发展和科技的进步，人类步入了大数据时代，大数据作为当前高科技时代的产 物，它的种类多而繁杂。如果想要得到需要的数据，则这些需要的数据有可能来源于多个不同的数据源中，此时，我们可以将多个数据源进行合并操作，从而获取到所需要的数据。

- **不一致数据的转换**

不一致数据转换主要是将不同业务系统中的相同类型的数据进行统一，例如同一个供应商在结算 系统的编码是XX0001，而在CRM（客户关系管理系统）中编码是YY0001，这样就需要将这两个业务系统中的数据抽取过来进行统一转换，转换成同一 个编码。

数据转换

- **数据粒度的转换**

业务系统一般存储非常明细的数据，而数据仓库中数据是用来分析的，不需要非常明细的数据。 一般情况下，会将业务系统数据按照数据仓库粒度进行聚合，这个过程就被称为数据粒度的转换。例如，将城市转换成省份或者直辖市。

- **数据的商务规则计算**

不同的企业，有着不同的业务规则和数据指标，这些指标应该计算完存储到数据仓库中，供企业决策者进行分析，从而得出战略性的企业决策。 例如，A公司和B公司的总公司想要得知各省份的手机日销售额，这就属于一个商务规则。

数据加载



数据的预处理过程，除了包括数据抽取、数据本身的清洗与检验以及数据转换操作，还包括数据加载操作，数据加载是数据预处理过程的最后一个步骤，主要是负责将清洗检验、转换后的高质量数据加载到目标数据库中。

数据加载

• 数据的加载机制

数据的加载机制与数据的抽取机制相类似，数据的加载机制可以分为全量加载和增量加载。其中，全量加载是指将目标数据表中的数据全部删除后，进行数据加载的操作；而增量加载是指目标表只加载源数据表中变化的数据，其中变化的数据包含新增、修改和删除的数据。

• 数据的商务规则计算

通常情况下，对于几千条甚至几十万条记录的数据迁移而言，采取DML（即数据操纵语言）的INSERT语句能够很好地将数据迁移到目标数据库中。然而，当数据迁移量过于庞大时，就不能使用INSERT语句，因为执行INSERT、UPDATE以及DELETE语句的操作都会生成事物日志，事物日志的生成会减慢加载的速度，故需要针对数据采取批量加载操作。

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY

目 录



1 预处理工具

2 数据预处理

3 示例

数据加载

```python
# 读取 CSV 文件
csv_file = 'data.csv'
df_csv = pd.read_csv(csv_file)
print("CSV 数据：")
print(df_csv.head())

# 读取 Excel 文件
excel_file = 'data.xlsx'
df_excel = pd.read_excel(excel_file, sheet_name='Sheet1')  # 指定工作表
print("\nExcel 数据：")
print(df_excel.head())

# 读取 JSON 文件
json_file = 'data.json'
df_json = pd.read_json(json_file)
print("\nJSON 数据：")
print(df_json.head())

# 连接到 SQLite 数据库并读取数据
sqlite_file = 'data.db'
conn = sqlite3.connect(sqlite_file)
df_sqlite = pd.read_sql_query("SELECT * FROM table_name", conn)
print("\nSQLite 数据：")
print(df_sqlite.head())
```

数据清洗

```python
# 1. 去除重复值
df_cleaned = df.drop_duplicates(subset=['Name'])
print("\n去除重复后的数据：")
print(df_cleaned)

# 2. 填充缺失值
# 填充 'Age' 和 'Salary' 列中的缺失值
df_cleaned['Age'] = df_cleaned['Age'].fillna(df_cleaned['Age'].mean())  # 使用均值填充缺失的
df_cleaned['Salary'] = df_cleaned['Salary'].fillna(df_cleaned['Salary'].median())  # 使用中位
print("\n填充缺失值后的数据：")
print(df_cleaned)

# 3. 处理异常值
# 假设 'Salary' 中薪资大于 100000 为异常值，设置为 NaN
df_cleaned['Salary'] = df_cleaned['Salary'].apply(lambda x: np.nan if x > 100000 else x)
print("\n处理异常值后的数据：")
print(df_cleaned)

# 4. 转换数据类型
# 将 'Age' 列转换为整数类型
df_cleaned['Age'] = df_cleaned['Age'].astype(int)
print("\n转换数据类型后的数据：")
print(df_cleaned)

# 5. 根据条件筛选数据
# 选择 'Age' 大于等于 30 的行
df_filtered = df_cleaned[df_cleaned['Age'] >= 30]
print("\n筛选后的数据（Age >= 30）：")
print(df_filtered)
```